



0106 AI 모델 개발 준비

AI 개발 세팅

Miniforge3 설치

■ Miniforge3

- conda 패키지 관리자의 최소 설치 프로그램
- 처음부터 conda-forge 채널을 기본으로 사용하도록 설정되어 있어 오픈 소스 중심의 최신 패키지 설치가 용이

구분	venv (+ pip)	Miniforge
주요 용도	단순한 파이썬 프로젝트, 웹 개발	데이터 사이언스, AI/머신러닝, 복잡한 연산
파이썬 버전	시스템에 설치된 버전만 사용 가능	환경마다 다른 파이썬 버전 설치 가능
비-파이썬 의존성	시스템(OS)에 직접 설치해야 함	C++, CUDA 등 외부 라이브러리 자동 설치
설치 속도	가벼움 (매우 빠름)	무거움
라이브러리 출처	PyPI (pip)	conda-forge (커뮤니티 관리 저장소)



huggingface API key : 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

kaggle API key : KGAT_81f0ca7e0f2b5505d6b1010284dd6493

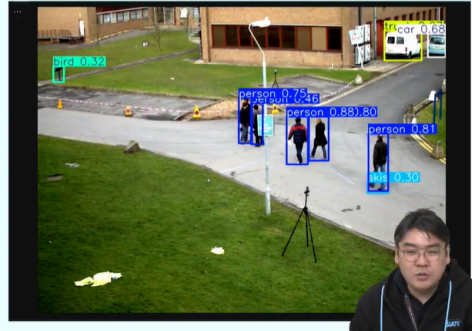
AI 모델 유형

컴퓨터 비전

- 기계가 영상을 이해하고 분석하는 AI 기술 분야
- 이미지 분류, 객체 탐지, 얼굴 인식, 이미지 분할

[실습]

- 객체 탐지 (Object Detection)
 - YOLO 이미지 객체 탐지
 - YOLO 영상 객체 탐지
 - YOLO Fine-Tuning
- OCR
 - EasyOCR 문자 인식
 - TrOCR Fine-Tuning

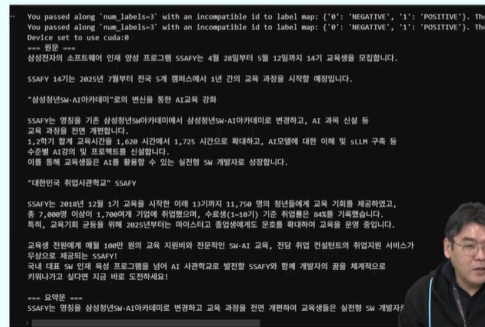


자연어 처리

- 인간의 언어를 컴퓨터가 이해하고 생성할 수 있게 하는 AI 기술
- 텍스트 분류, 감정 분석, 기계 번역, 요약, STT (Speech To Text), TTS (Text To Speech)

[실습]

- 텍스트
 - BART/BERT 활용 텍스트 감정 분석/요약/분류
 - 텍스트 요약 Fine-Tuning
 - 문서 분류 Fine-Tuning
- 음성
 - Whisper STT (Speech To Text)
 - Coqui-TTS + XTTSv2 TTS (Text to Speech)

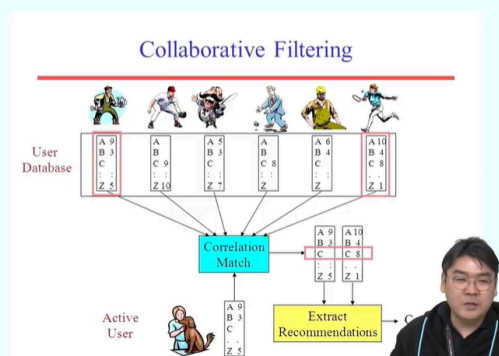


추천

- 사용자의 선호도와 패턴을 분석
- 관련성 높은 상품, 콘텐츠, 서비스 등을 제안하는 기술

[실습]

- 협업 필터링
 - 협업 필터링 기반 추천
- 콘텐츠 기반 추천
 - TF-IDF 기반 콘텐츠 추천
 - PCA, LSA, TF-IDF, K-means 활용 추천 모델 개발



예측

- 과거 데이터 패턴을 분석해서 미래 결과나 동향을 추정
- 주식시장 예측, 수요예측, 질병예측, 날씨예보

[실습]

- 시계열 예측
 - ARIMA 시계열 예측
- 이상탐지
 - Isolation Forest 이상탐지



LLM(거대 언어 모델)

- 방대한 텍스트 데이터로 사전 학습된 초거대 AI 모델
- 텍스트 생성, 대화, 번역, 요약 등 다양한 언어 작업을 수행

[실습]

- 질의 응답
 - Llama활용 질의응답
 - Llama활용 질의응답 Fine-Tuning
- 도메인 특화 Chatbot
 - Llama활용도메인 특화 Chatbot
 - Llama활용도메인 특화 Chatbot Fine-Tuning

[illegible]

AI 기술 개요

인공지능(AI)

- 인간의 지능을 모방해 학습, 추론, 문제 해결 등을 수행할 수 있는 컴퓨터 시스템을 개발하는 컴퓨터 과학의 한 분야
- 인간의 지능적인 행동을 기계로 구현하는 모든 시도를 포함.
- 논리적 추론, 탐색, 계획, 학습, 자연어 처리, 컴퓨터 비전, 음성인식 등이 포함됨.
- 기능적 분류
 - 약인공지능 : 특정 작업을 수행하도록 설계되고 훈련된 AI
 - 강인공지능 / 일반 인공지능(AGI) : 인간과 대등하거나 그 이상의 지적 능력을 갖춘 가상의 AI
 - * 초인공지능(Artificial Super Intelligence, ASI) : 인간의 지능을 모든 면에서 능가하는 가상의 단계

머신러닝(ML)

- 컴퓨터가 명시적으로 프로그래밍 되지 않고 데이터로부터 스스로 학습하여 패턴을 인식
- 이를 기반으로 예측이나 결정을 내릴 수 있도록 하는 알고리즘과 기술들
- 데이터의 품질과 데이터로부터 추출한 '특성(Feature)'의 적절성에 의존
- 구조화된 데이터(Structured Data)에서는 효율적
- 이미지나 음성 같은 비구조화 데이터(Unstructured Data)를 처리하는 데는 한계가 있음

딥러닝(DL)

- 머신러닝의 한 분야
- 인간의 뇌 신경망 구조에서 영감을 받은 인공신경망(ANN) 중 특히 여러 개의 은닉층을 가진 심층신경망(DNN)을 사용하여 데이터로부터 복잡하고 추상적인 특징(feature)을 계층적으로 자동 추출하고 학습하는 기술
- 비정형 데이터 처리에 강점
- 데이터 양이 늘어날수록 성능이 지속적으로 향상
- 대표적인 기술
 - 합성곱 신경망(CNN)
 - 순환 신경망(RNN)
 - 생성적 적대 신경망(GAN)
 - 트랜스포머(Transformer)

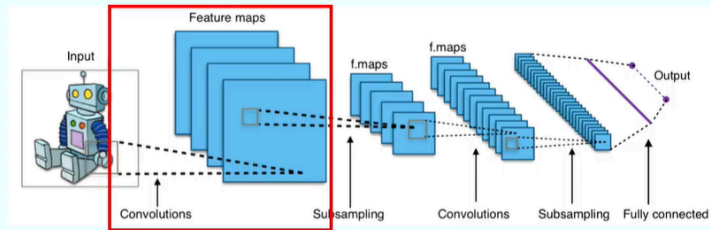


CNN

CNN(Convolutional Neural Network) 이란?

- 이미지나 영상과 같은 격자형 데이터의 특징을 자동으로 학습하는데 최적화된 신경망
- 사람의 능력을 모방하여 이미지의 작은 구역을 훑으며 특징을 추출하는 방식으로 작동
- 단계
 - 특징 추출(Feature Extraction) 단계
 - : 합성곱(Convolution)과 풀링(Pooling/Subsampling)을 반복하며 이미지의 주요 특징을 잡아냄
 - 분류(Classification) 단계
 - : 추출된 특징을 바탕으로 최종 판단
- 주요 특징 및 장점
 - 공간적 계층 구조의 학습
 - 가중치 공유 '**와 **' 국소 수용 영역

CNN Architecture

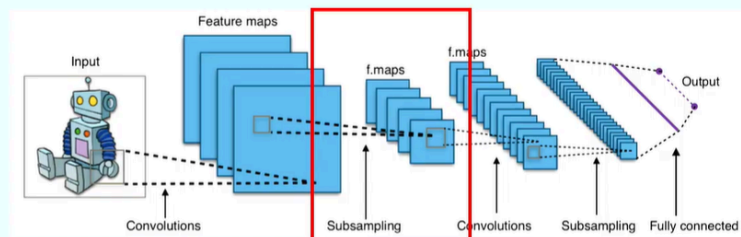


<그림 3> CNN 아키텍처 (출처 : https://en.m.wikipedia.org/wiki/File:Typical_cnn.png)

- Convolutions(합성곱)

- 이미지에서 특징을 추출하는 핵심 과정
- 작은 필터가 이미지를 훑으면서 합성곱 연산을 수행하여 이 결과가 특징맵(Feature maps)으로 불리는 새로운 이미지 형태로 나타남.
- 여러 개의 필터를 사용하면 다양한 특징을 동시에 추출할 수 있음(모서리, 질감, 색상 등)

CNN Architecture

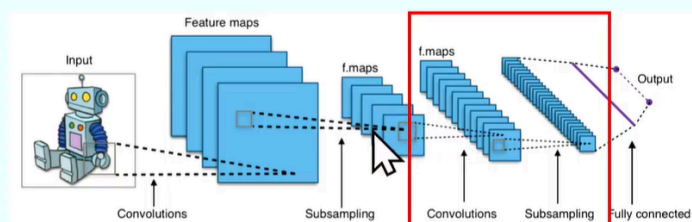


<그림 3> CNN 아키텍처 (출처 : https://en.m.wikipedia.org/wiki/File:Typical_cnn.png)

- Subsampling(서브샘플링)

- 특징 맵의 크기를 줄여(다운샘플링) 계산량을 줄이고, 과적합을 방지하는 역할을 함
- 최대 풀링(Max pooling)이나 평균 풀링(Average Pooling) 방식이 사용 됨

CNN Architecture

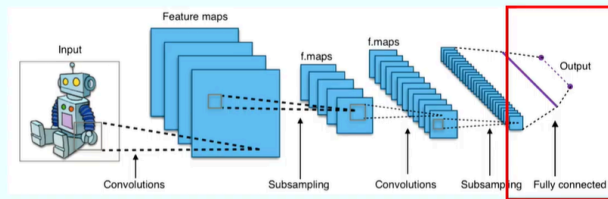


<그림 3> CNN 아키텍처 (출처 : https://en.m.wikipedia.org/wiki/File:Typical_cnn.png)

- Feature Extraction(반복)

- 초반 단계: 점, 선, 경계선 같은 낮은 수준의 특징을 추출
- 후반 단계: 눈, 코, 로봇의 손등과 같은 복잡하고 높은 수준의 특징을 결합하여 추출
- 반복을 통해 층을 쌓을 수록 더 추상적이고 고차원적인 정보를 이해
- 반복회수에 따라 기울기 손실(Vanishing Gradient), 과적합(Overfitting) 등의 문제가 발생할 수 있음

CNN Architecture



<그림 3> CNN 아키텍처 (출처 : https://en.m.wikipedia.org/wiki/File:Typical_cnn.png)

- Fully Connected(완전 연결)

- 여러 합성곱과 서브샘플링 단계를 거쳐 추출된 특징 맵들은 1차원 배열로 평탄화(Flatten)된 후 완전 연결계층에 입력됨
- 추출된 특징들을 바탕으로 이미지를 최종적으로 분류하거나 예측하는 작업이 이루어 짐



활용 예시

의료 영상 분석

<그림 4> 흉급 환자 분류 솔루션
(출처 : <https://m.dongascience.com/news.php?id=69203>)

객체 탐지 & 얼굴 인식

Classification

CAT

Classification + Localization

CAT

Object Detection

CAT, DOG, DUCK

<그림 5> 이미지 분류, 객체 탐지 (출처 : <https://www.geeksforgeeks.org/what-is-image-classification/>)

Face Recognition

Emotion Recognition

<그림 6> 얼굴 인식, 감정 (출처 : <https://medium.com/analytics-vidhya/face-emotion-recognition-hands-on-guide-8e23f3d00254>)

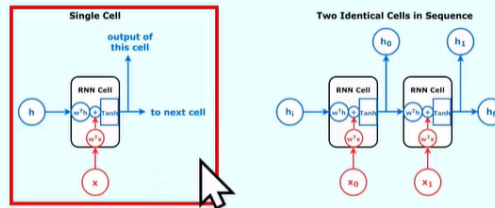
RNN

RNN(Recurrent Neural Network)이란?

- 시간적 흐름이 있거나 순서가 있는 데이터(시계열, 텍스트, 음성 등)를 다루기 위해 특화된 모델
- 기억(Memory)
 - : 이전 단계에서 계산된 정보(은닉 상태)를 다음 단계로 전달하여, 과거의 정보가 현재의 판단에 영향을 미침
- 은닉층이 다시 자신의 입력으로 들어오는 순환구조(Recurrent Structure)
- 주요 특징 및 작동 방식
 - 순차적 의존성 학습
 - 시퀀스 문맥 정보 포착



RNN Architecture

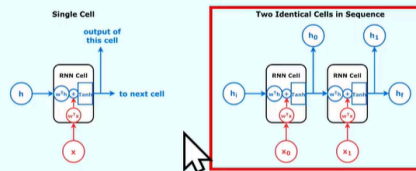


<그림 7> RNN 아키텍처 (출처 : https://en.m.wikipedia.org/wiki/File:RNN_architecture.png)

- RNN Cell

- RNN의 가장 기본적인 구성 요소
- 현재 시점의 입력 데이터와 이전 시점의 은닉 상태(hidden state)를 모두 받아 새로운 은닉 상태와 출력을 만들어 냄
- x (입력): 현재 시점의 데이터(예 : 문장의 현재 단어)
- h (은닉 상태): 이전 시점의 RNN 셀에서 전달되는 정보. 이전 데이터의 요약된 정보(Hidden State)를 담고 있어, RNN이 과거의 정보를 '기억'할 수 있게 해 줌

RNN Architecture

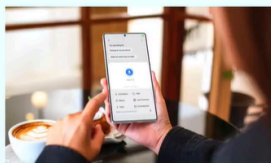


<그림 7> RNN 아키텍처 (출처 : https://en.m.wikipedia.org/wiki/File:RNN_architecture.png)

- Two Identical Cells in Sequence (시간 순서로 펼친 구조)
- 시계열 데이터($x_0, x_1, x_2, \dots$)를 시간 순서대로 RNN Cell에 입력하면 가중치(w)를 곱하여 Tanh 함수로 비선형성을 추가
- 정보 전달
- 은닉 상태(h_0, h_1)가 다음 Cell로 전달되어 시퀀스 전체에 걸쳐 문맥 정보를 유지하고 순차적 패턴을 학습
- 시퀀스 예측
- 매 시점마다 출력을 생성하거나 최종 은닉 상태를 사용하여 텍스트 생성, 번역, 시계열 예측 등의 작업 수행

활용 예시

음성 인식

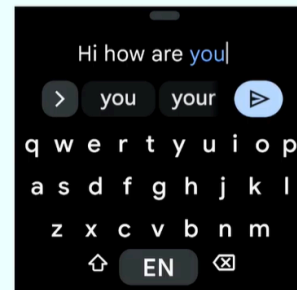


Galaxy AI
(출처 : <https://www.samsung.com/sec/galaxy-ai/>)



<그림9> 유튜브 자동 자막 생성 (출처 : <https://www.youtube.com/watch?v=0yLijvFFwN8>)

텍스트 자동 완성



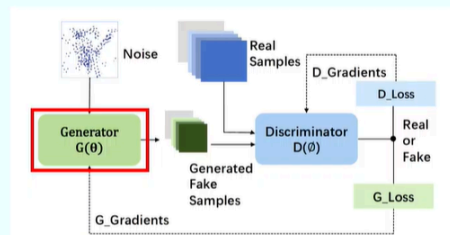
<그림 10> Google Gboard 키보드 앱 (출처 : <https://play.google.com/store/apps/details?id=com.google.android.inputmethod.latin>)

GAN

GAN(Generative Adversarial Networks) 이란?

- 두 개의 신경망이 서로 경쟁(adversarial)하며 발전하는 독특한 생성 모델
- 생성자(Generator) 네트워크와 판별자(Discriminator) 네트워크로 구성되며, 이 두 네트워크가 마치 경쟁하는 게임처럼 학습

GAN Architecture

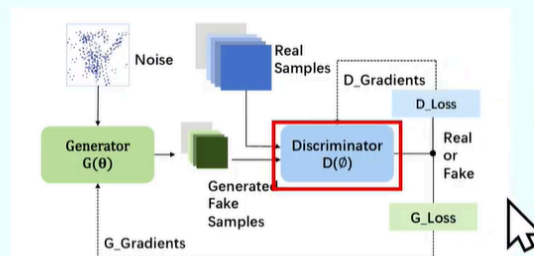


<그림 11> GAN 아키텍처 (출처 : https://www.researchgate.net/figure/Typical-GAN-architecture_fig2_385595201)

- Generator (생성자, G)

- 실제 데이터와 유사한 가짜 데이터를 만드는 역할
- 무작위의 노이즈(Noise)를 입력으로 받아, 이를 실제 데이터처럼 보이는 가짜 샘플(Generated Fake Samples)로 변환
- 생성자는 판별자를 속이는 것을 목표로 학습

GAN Architecture

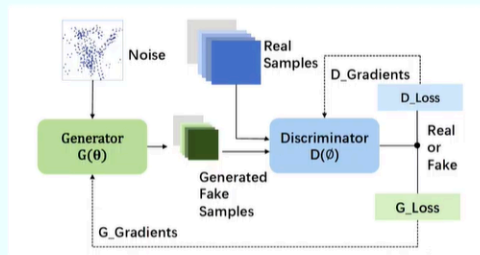


<그림 11> GAN 아키텍처 (출처 : https://www.researchgate.net/figure/Typical-GAN-architecture_fig2_385595201)

- Discriminator (판별자, D)

- 입력된 데이터가 실제 데이터인지, 아니면 생성자가 만든 가짜 데이터인지 구별하는 역할
- 판별자는 실제 샘플(Real Samples)과 생성된 가짜 샘플을 모두 입력받아, 각각을 'Real' 또는 'Fake'로 분류
- 판별자는 생성자가 만든 가짜 데이터를 정확하게 구분하는 것을 목표로 학습

GAN Architecture



<그림 11> GAN 아키텍처 (출처 : https://www.researchgate.net/figure/Typical-GAN-architecture_fig2_385595201)

- Adversarial Training (적대적 학습)

- 판별자 학습

판별자는 실제 샘플을 'Real'로, 가짜 샘플을 'Fake'로 정확하게 분류하도록 학습
이때 발생하는 손실(D_Loss)을 줄이는 방향으로 D_Gradients를 사용해 모델의 가중치를 업데이트

- 생성자 학습

생성자는 판별자를 속여서 자신의 가짜 샘플이 'Real'로 분류되도록 학습
생성자의 목표는 판별자가 자신의 가짜 샘플을 'Fake'라고 판별하지 못하게 만드는 것
이 과정에서 G_Loss가 계산되고, G_Gradients를 통해 생성자의 가중치가 업데이트

Transformer

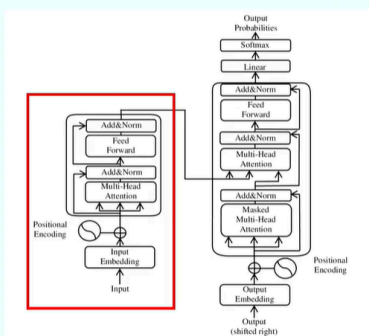
Transformer란?

- 2017년 구글 연구진이 발표한 "Attention is All You Need" 논문에서 등장한 혁신적인 신경망 구조
- 순차적인 처리 방식을 버리고 어텐션(Attention) 메커니즘을 중심으로 시퀀스 데이터를 처리하는 것이 특징

- 주요 특징 및 장점

- Self-Attention 메커니즘순차적 의존성 학습
- 장거리 의존성 학습
- 확장성

Transformer Architecture



<그림 13> Transformer 아키텍처
(출처 : https://www.researchgate.net/figure/Typical-GAN-architecture_fig2_385595201)

- Encoder (인코더)

- 입력 문장을 받아 문맥을 이해하고 그 정보를 벡터로 변환하는 역할

- 입력처리

입력 문장의 단어들은 숫자 벡터로 변환됨(Input Embedding).

단어의 순서 정보를 학습하기 위해 각 위치에 해당하는 위치 인코딩(Positional Encoding) 값을 벡터에 더해줌

- 문맥 파악

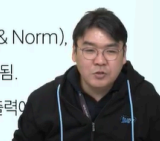
멀티 헤드 어텐션(Multi-Head Attention) 계층이 작동하며, 문장 내의 모든 단어들 간의 관계를 분석

- 정보 통합

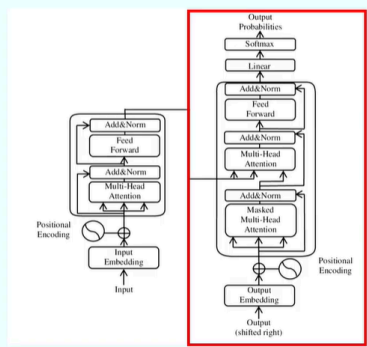
어텐션으로 얻은 문맥 정보가 원래의 단어 벡터에 더해지고 (Add & Norm),

피드 포워드(Feed Forward) 신경망을 거쳐 새로운 정보가 통합됨.

이 과정이 여러 층 반복되면서 입력 문장의 복잡한 의미가 인코더 출력에



Transformer Architecture



<그림 13> Transformer 아키텍처
(출처: https://www.researchgate.net/figure/Typical-GAN-architecture_fig2_385595201)

- Decoder (디코더)

- 인코더의 출력 정보를 바탕으로 새로운 문장을 생성하는 역할

- 입력 처리:

디코더는 문장을 한 단어씩 순차적으로 생성

이전에 생성된 단어들이 Masked Multi-Head Attention 계층의 입력으로 들어감.

- 문맥 연결

두 번째 어텐션 계층에서는 인코더의 출력 정보와 디코더의 현재 입력 정보가 연결됨

디코더는 이 과정을 통해 인코더가 파악한 입력 문장의 문맥을 참고하여 적절한 단어를 예측

- 단어 생성

최종적으로, 선형(Linear) 계층과 소프트맥스(Softmax) 함수를 거쳐 다음 단어로 쓸 수

있는 모든 단어들의 확률을 계산하고, 가장 높은 확률을 가진 단어를 출력

이 과정이 문장의 끝을 나타내는 단어가 나올 때까지 반복



활용 예시

AI 어시스턴트

Message ChatGPT

Search

<그림 14> ChatGPT (출처: <https://openai.com/index/introducing-chatgpt-search/>)

How can I help you this morning?

<그림 15> Claude (출처: <https://claude.ai/download>)

AI 에이전트

공식 컬렉션에서의 사용 사례

Manus

<그림 17> Manus Use case gallery, 출처: <https://manus.im/usecase-collection>



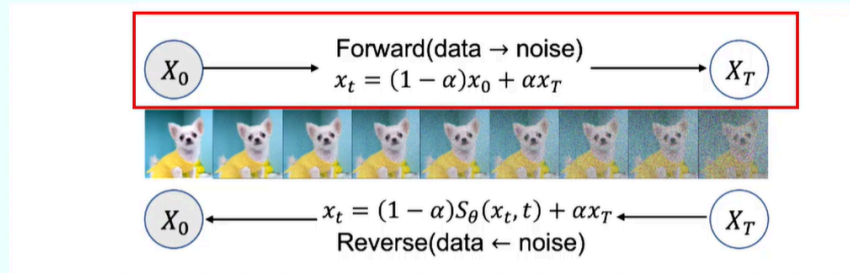
Diffusion Model

Diffusion Model이란?

- 점진적인 노이즈 추가 및 제거 과정을 통해 데이터를 생성하는 가장 최신의 생성 모델

- 사진이 점점 흐려지는 과정을 거꾸로 뒤집어 흐릿한 노이즈에서 선명한 이미지를 얻는 원리와 유사

Diffusion Model Architecture



<그림 18> Diffusion Model 아키텍처 (출처 : <https://www.digitalocean.com/community/tutorials/denoising-via-diffusion-model>)

- Forward 과정 : 원본 데이터(X_0)에서 시작해 단계적으로 노이즈를 추가하며, 완전한 노이즈(X_t)로 변환
- Reverse 과정 : 신경망(S_θ)이 현재 노이즈 샘플(X_t)를 입력 받아 노이즈를 예측하고, 이전 단계의 샘플(X_{t-1})로 복원하는 방법을 활용
- 데이터 생성 : 학습된 Reverse 과정을 통해 랜덤 노이즈에서 시작하여 신경망이 점진적으로 노이즈를 제거하며 고품질 데이터 생성

활용 예시

이미지/비디오 생성

<그림 19> Midjourney(출처 : <https://the-decoder.com/midjourney-new-image-tool-works-in-reverse/>)

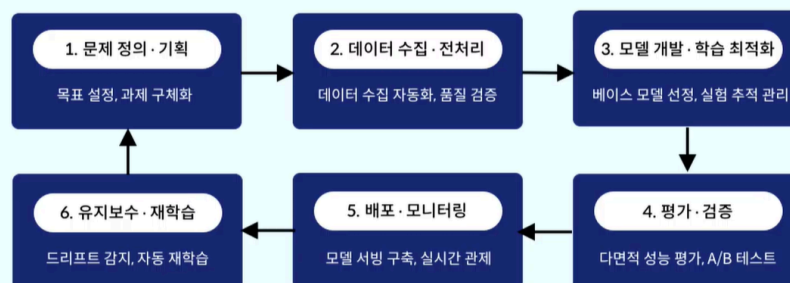
<그림 15> Runway Gen-4 (출처 : <https://www.youtube.com/watch?v=br9b3-cxTPQ>)

음악 생성

<그림 21> Suno AI, 출처 : <https://www.zdnet.com/article/how-to-generate-your-own-music-the-ai-powered-suno/>)

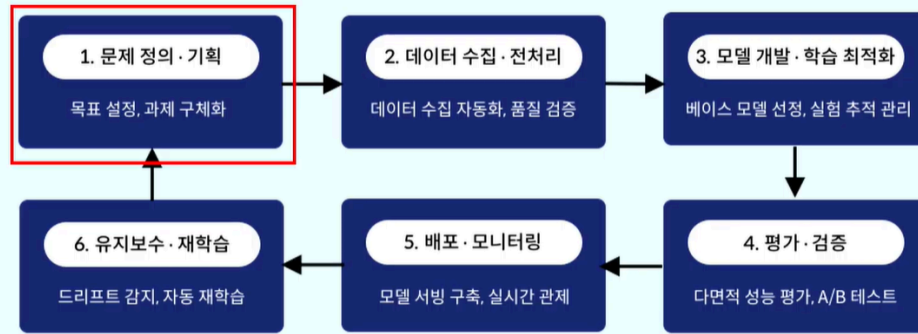
AI 개발 라이프사이클

AI모델의 전체 생명 주기를 체계적으로 관리하는 프로세스



- 문제 정의와 기획으로 시작하여 데이터 수집·전처리를 거쳐 모델을 개발하고 학습을 최적화한 후, 다양한 지표로 평가
- 검증된 모델은 실제 환경에 배포되어 지속적으로 모니터링하며, 성능 저하 시 재학습을 통해 개선하는 순환 구조

AI모델의 전체 생명 주기를 체계적으로 관리하는 프로세스

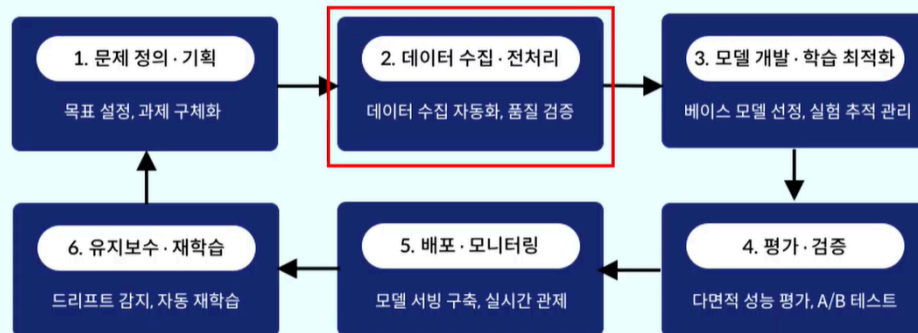


목표 설정과 과제의 구체화

기술적 타당성 검토와 도메인 선정

데이터 전략 및 자원 계획 수립

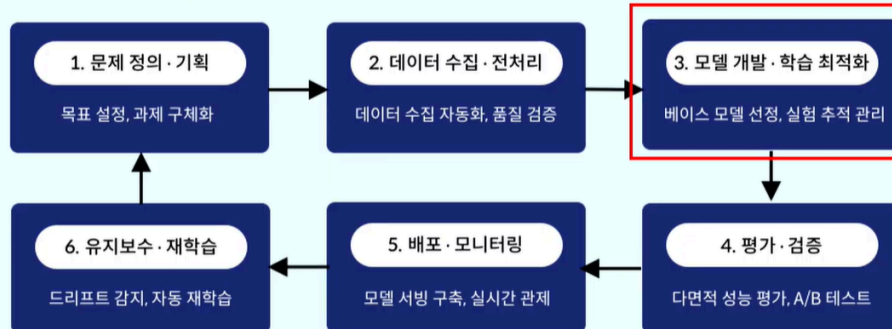
AI모델의 전체 생명 주기를 체계적으로 관리하는 프로세스



데이터 수집의 다각화와 품질 관리

도메인 특화 전처리 기술 심층 분석

AI모델의 전체 생명 주기를 체계적으로 관리하는 프로세스

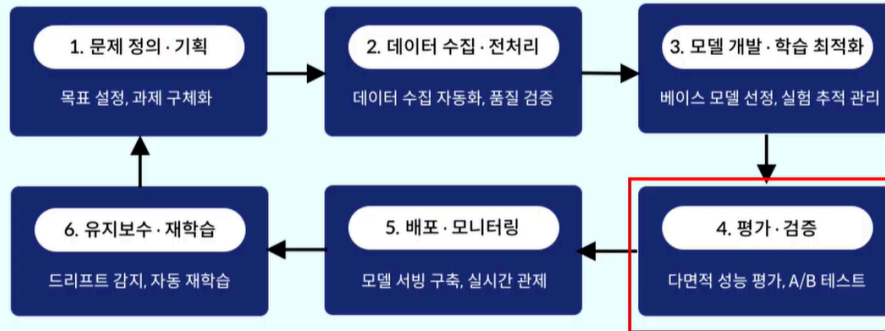


- 베이스 모델 선정 및 아키텍처 설계

- 학습 전략 및 미세 조정(Fine-Tuning)

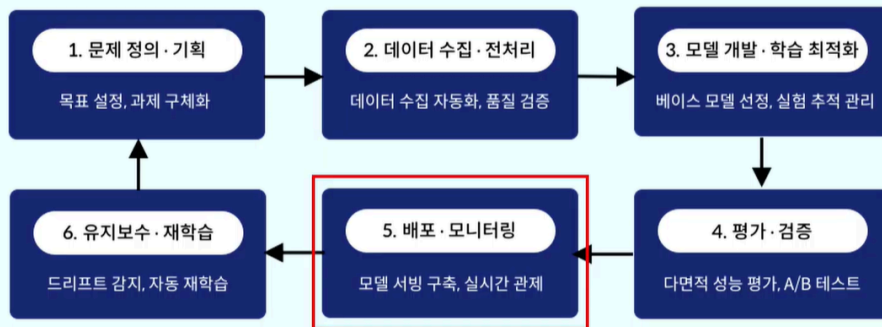
- 하이퍼파라미터 튜닝 및 최적화

AI모델의 전체 생명 주기를 체계적으로 관리하는 프로세스



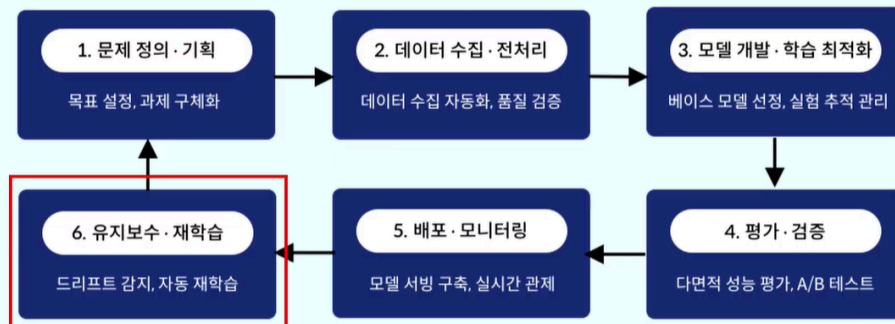
- 성능 평가의 다차원적 접근
- 검증 방법론 및 오류 분석
- 교차검증(Cross-Validation), 오류 분석(Error Analysis), A/B테스트 등

AI모델의 전체 생명 주기를 체계적으로 관리하는 프로세스



- 모델 서빙(Model Serving) 아키텍처 구축 - 실시간 추론, 배치추론, 엣지배포
- 인프라 및 컨테이너 오케스트레이션 - Docker 컨테이너 기술이 표준
- 지속적인 모니터링 및 로깅 - 시스템 모니터링, 모델 성능 모니터링, 데이터 드리프트 (Data Draft) 감지

AI모델의 전체 생명 주기를 체계적으로 관리하는 프로세스



- 모델 성능 저하(Model Decay)와 데이터 드리프트, 콘셉트 드리프트 대응
- 자동화된 MLOps 파이프라인 (CI/CD/CT)
- 데이터 버전 관리 (Data Version Control)

AI가 가져온 역할 분담의 변화

AI 이전	AI 이후
FE 개발자 : UX/UI 구현	AI FE 개발자 : 이상호작용 UI + 기존 UX/UI
BE 개발자 : 비즈니스 로직	AI BE 개발자 : AI 파이프라인(추론, 벡터 검색, RAG 등) 통합 + 기존 BE
DBA : 데이터베이스 관리(스키마, SQL 등)	데이터 엔지니어 : AI데이터 파이프라인 (대용량 학습 데이터, 벡터 DB 및 임베딩 관리 등) + 기존 DB
DevOps : 배포, 인프라, 모니터링	MLOps : 모델 생명 주기 관리 + 기존 DevOps
-	AI/ML 엔지니어 : 도메인 별 모델 리서치, 모델 성능 튜닝 및 평가
-	프롬프트 엔지니어 : 도메인 별 모델 리서치, 모델 성능 튜닝 및 평가



개발 환경 설정

Python 가상환경 설정

- 프로젝트 별 패키지 의존성을 분리하고, 개발 환경 설정의 편의를 위해 가상환경을 설정
- 가상환경은 프로젝트별로 복잡하게 엮일 수 있는 의존 패키지들을 격리할 수 있는 이점을 가짐
 - 프로젝트 마다 다른 라이브러리 버전의 공존이 한PC에서 가능
 - 시스템 파이썬을 오염시키지 않아 운영체제나 다른프로젝트와 충돌방지
 - 환경 별 의존 패키지 목록 requirements.txt등 으로 손쉽게 패키지 재설치 가능

- Miniforge Prompt 실행



- 자동 환경 변수 설정 진행

conda init

```
Miniforge Prompt - conda create -n ai_env python3.10

C:\Users\ce0\workspace\ai-edu-dev>conda init
no change C:\Users\ce0\Miniforge3\Scripts\conda.exe
no change C:\Users\ce0\Miniforge3\Scripts\conda-env.exe
no change C:\Users\ce0\Miniforge3\Scripts\conda-script.py
no change C:\Users\ce0\Miniforge3\Scripts\conda-env-script.py
no change C:\Users\ce0\Miniforge3\condabin\conda.bat
no change C:\Users\ce0\Miniforge3\Library\bin\conda.bat
no change C:\Users\ce0\Miniforge3\condabin\conda_activate.bat
no change C:\Users\ce0\Miniforge3\condabin\conda_rename_tmp.bat
no change C:\Users\ce0\Miniforge3\condabin\conda_auto_activate.bat
no change C:\Users\ce0\Miniforge3\condabin\conda_hook.bat
no change C:\Users\ce0\Miniforge3\Scripts\activate.bat
no change C:\Users\ce0\Miniforge3\condabin\activate.bat
no change C:\Users\ce0\Miniforge3\condabin\deactivate.bat
modified C:\Users\ce0\Miniforge3\Scripts\activate
modified C:\Users\ce0\Miniforge3\Scripts\deactivate
modified C:\Users\ce0\Miniforge3\Scripts\fish_conda.fish
no change C:\Users\ce0\Miniforge3\shell\condabin\conda.ps1
modified C:\Users\ce0\Miniforge3\shell\condabin\conda-hook.ps1
no change C:\Users\ce0\Miniforge3\Library\site-packages\conda\contrib\conda.xsh
modified C:\Users\ce0\Miniforge3\Scripts\profile.d\conda.csh
modified C:\Users\ce0\Documents\WindowsPowerShell\profile.ps1
modified HKEY_CURRENT_USER\Software\Microsoft\Command Processor\AutoRun

==> For changes to take effect, close and reopen your current shell. $==
```

- “ai_env”라는 Python 3.10버전 기반의 가상환경 생성

```
conda create -n ai_env python=3.10
```

- 설치 완료 후 설치한 ai_env 가상환경 활성화

```
conda activate ai_env
```

- python 입력해서 Python 버전 확인

- exit()을 입력하여 Python 프롬프트 종료

- 가상환경 관리 명령어

#가상 환경 활성화

```
conda activate ai_env
```

#가상 환경 비활성화

```
conda deactivate
```

#가상 환경 삭제

```
conda env remove -n ai_env
```

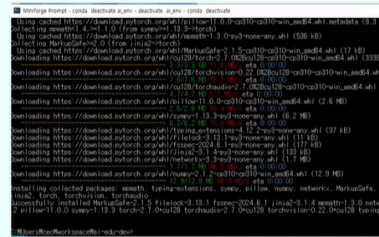
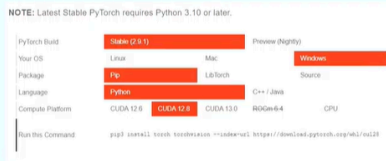
#가상 환경 목록 조회

```
conda env list
```

Pytorch

PyTorch

1. <https://pytorch.org/get-started/locally/> 접속
2. 지원하는 CUDA 버전 선택(12.8) 및 설치 명령어 생성



3. 가상환경(ai_env)이 활성화 된 상태에서 생성한 명령어 실행하여 PyTorch 설치

```
#가상 환경 활성화
conda activate ai_env
# PyTorch 설치
pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu128
```

공통 필수 라이브러리 설치

1. numpy, panda, matplotlib 설치

```
conda install numpy pandas matplotlib
```

2. opencv, scikit-learn 설치

```
conda install -c conda-forge opencv scikit-learn
```

3. transformers, datasets 설치(pip 사용)

```
pip install transformers datasets
```

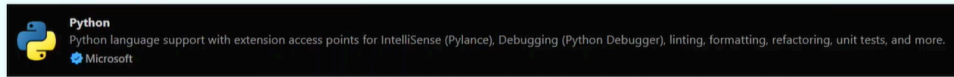
4. jupyter, ipykernel 설치(pip 사용)

```
pip install jupyter ipykernel
```

라이브러리	특징 및 주요 기능
NumPy	- Python의 대표적인 과학 연산 라이브러리 - 대규모 배열 및 행렬 연산을 빠르고 효율적으로 수행
Pandas	- 데이터 분석의 필수 도구 - 행렬 형태의 DataFrame 구조를 사용하여 데이터를 가공하고 조작하는 데 사용
Matplotlib	- 가장 기본적인 시각화 라이브러리 - 모델의 학습 곡선이나 분석 결과를 차트와 그래프로 시각화.
scikit-learn	- 머신러닝 알고리즘을 집대성한 라이브러리 - 분류, 회귀, 군집화 등 다양한 분석 모델을 손쉽게 구현

VSCode Extension 설치

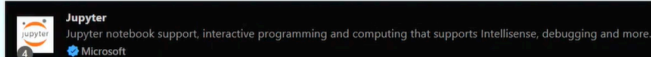
- Python : Python 언어 지원 및 IntelliSense 제공. 가상환경 선택, 코드 실행, 디버깅, Jupyter Notebook 기능까지 포함하는 핵심 Extension



- Python Data Science Extension Pack: 데이터 과학 도구 모음 Extension



- Jupyter: VSCode에서 Jupyter Notebook(.ipynb)을 실행하고 편집할 수 있게 해주는 Extension

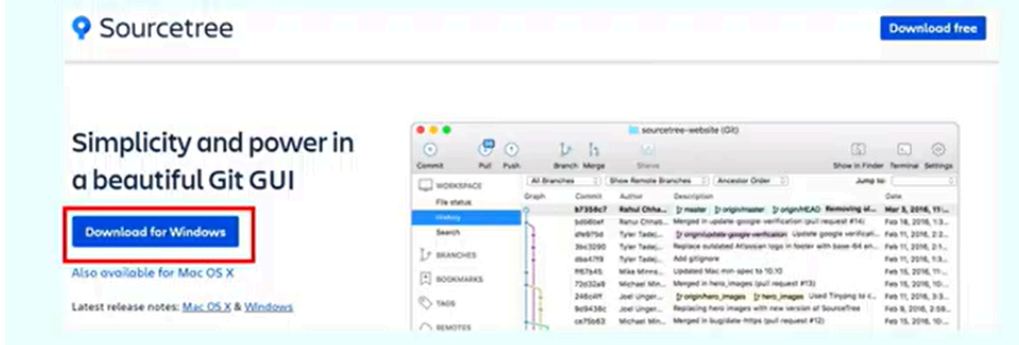


Sourcetree 설치

- Git을 위한 GUI 클라이언트

1. <https://www.sourcetreeapp.com/> 에 접속

2. 다운로드 및 설치



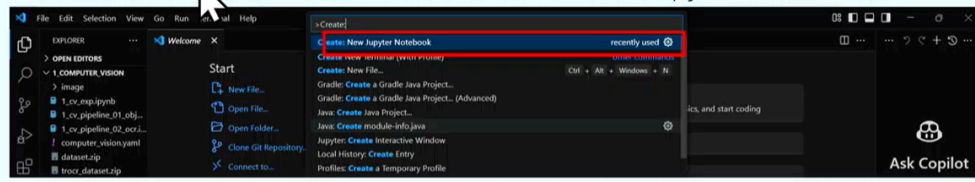
이건 선택사항

Jupyter Notebook

- 브라우저 기반 대화형 컴퓨팅 환경: 라이브 코드, 수식, 시각화 자료 및 설명 텍스트를 하나의 문서에 통합하여 작성
- 데이터 과학 분야에서 표준 도구 : 데이터 분석과 과학적 연구 결과를 기록하고 공유하는 데 최적화
- 핵심 기능
 - 셀(Cell) 단위 실행: 코드를 블록 단위로 나누어 실행하고 결과를 즉시 확인 가능
 - 풍부한 문서화: 마크다운(Markdown)을 지원하여 코드, 텍스트, 수식, 이미지를 하나로 통합
 - 다양한 언어 지원 : 주로 Python과 함께 사용되지만 R, Julia 등 40개 이상의 다양한 언어 지원
 - 데이터 시각화 최적화: 그래프와 차트를 코드 바로 아래에 출력하여 직관적인 분석 가능.

VSCode에서 Jupyter 생성

1. Command Palette (Ctrl + Shift + P) 에서 Create : new Jupyter Notebook 선택



2. 우측 상단 select Kernel 클릭하여 사전에 구성한 Python 가상환경인 ai_env를 지정

